

PYTHON: basics

- **int, float:** цілі числа і числа з плаваючою
комою

int&float: вбудовані числові типи

python має декілька вбудованих числових типів. В межах цього курсу ми познайомимось лише з **int** (цілі числа) та **float** (числа з плаваючою комою).

Ціле число — це ціле число без десяткових знаків. Наприклад, 1 є цілим числом, але 1.0 не є таким. Цілочисельний тип даних має назву **int**, яку можна побачити за допомогою type():

```
>>> type(1)
<class 'int'>
```

Зверніть увагу що в коді саме число - зображується без лапок. І як функція **str()** - перетворює число в рядок, так же існує функція **int()** - яка приймає рядок і якщо це можливо - перетворює його в число:

* - зверніть увагу в документації на можливість роботи не лише з десятковими числами, а і з двійковими, восьмирічними і шіснадцятирічними. Доречи - число може бути представлено рядком не лише арабськими цифрами.

```
>>> a = 42
>>> a
42
>>> a_string = str(a)
>>> a_string
'42'
>>> int(a_string)
42
```

int&float: вбудовані числові типи

для зручності набору ви можете при наборі чисел розбивати цифри на групи символами підкреслення

```
>>> 1000000
1000000
>>> 1_000_000
1000000
```

Як і цілі числа, числа з плаваючою точкою можна створити з літералів із плаваючою комою або шляхом перетворення рядка на число з плаваючою точкою за допомогою float():

```
>>> type(1.0)
<class 'float'>
>>> float('1.0')
1.0
```

Існує декілька шляхів представлення літералу з плаваючою комою. Кожен із наведених нижче елементів створює літерал із плаваючою комою зі значенням один мільйон:

```
>>> 1000000.0
1000000.0
>>> 1_000_000.0
1000000.0
>>> 1e6
1000000.0
```


int&float: математичні операції

додавання, виконується оператором “+”, операнди - не обов’язково одного типу. Але результат - має тип **int** лише якщо два операнда **int**:

```
>>> 1 + 2      # два операнда - int
      3         # int
>>> 1.0 + 2     # один операнд - float
      3.0       # float
```

оператор віднімання “-”, все інше аналогічно оператору додавання. А якщо у нього буде лише один операнд справа - то він просто змінює йому знак:

```
>>> a = - 3
>>> 0 - a
      3
>>> 0 - - a
     -3
```

щоб робити код більш читабельним і керувати черговістю виконання операцій - використовуй дужки, як в математиці:

```
>>> 2 + (-1) - (25 - (-15))
     -39
```

int&float: математичні операції

множення (тип результату - як в додаванні і відніманні):

```
>>> 4 * 5      # два операнда - int
      20        # int
>>> 4 * 5.0    # хоч один операнд - float
      20.0      # float
```

поділ (результат - завжди **float**)

```
>>> 10 / 5
      2.0
>>> 15 / 2
      7.5
```

показники ступеня (обидва операнда можуть бути як **int**, так і **float**, додатними і від'ємними), тип результату операції визначається як для додавання і віднімання. Якщо показник ступеню від'ємний - результат завжди float, бо від'ємний показник ступеню передбачає ділення одиниці на результат операції:

```
>>> 2 ** 4
      16
>>> 2 ** 4.5
      22.627416997969522
>>> 2 ** -4
      0.0625
>>> -2 ** 3
      -8
```

int&float: математичні операції

цілочисельне ділення (тип результату - як в додаванні і відніманні):

```
>>> 5 // 2
```

```
2
```

```
>>> 5.6 // 2
```

```
2.0
```

залишок від цілочисельного ділення (тип результату - як в додаванні і відніманні):

```
>>> 5 % 2
```

```
1
```

```
>>> 5.6 % 2.5
```

```
0.59999999999999996
```

в останньому прикладі два момент на які потрібно звернути увагу:

- ми можемо і цілочисельно ділити і отримувати залишок від такого ділення для будь-якого дільника - в останньому прикладі це **float**
- результат операції не є точним - ви розумієте що відповідь повинна бути 0.6, а ми маємо близьку відповідь - але не таку. Чому? Поговоримо на наступному слайді.

int&float: точність операцій

```
>>> 0.1 + 0.2  
0.30000000000000004
```

Ви бачите що це **“майже правильний”** результат. Чому так?

Це не помилка python, це помилка представлення чисел з плаваючою комою. Це пов'язано зі способом зберігання чисел з плаваючою комою в пам'яті комп'ютера.

як число $\frac{1}{3}$ в десятковому представленні є 0.333... з нескінченною кількістю знаків і обмеження кількості знаків для його запису одразу дасть приблизний результат. Точно так же у двійковому представленні $\frac{1}{10}$ (0.1 в десятковій системі счислення) це нескінченно повторюваний дріб. А так як комп'ютери мають обмежену пам'ять і для збереження чисел використовують кінцеву кількість байт - то такі числа зберігаються як наближення.

не використовуйте для опису грошових сум тип даних float. Для цього в python існують інші типи даних і інші підходи, з якими ви детальніше познайомитесь в майбутньому.

int&float: функції

В python є декілька вбудованих функцій для роботи з числами (найпоширеніші - **round()** - округлення, **abs()** - абсолютне значення, **pow()** - піднесення до деякого степеня). Також є вбудований модуль **math** - який надає велику кількість різноманітних математичних операцій. Детально пропоную розібратись за документацією. А ми розберемо роботу популярної функції **round()**

```
>>> round(2.6)
3          # тут зрозуміле округлення - до найближчого цілого
>>> round(3.5)
4          # округлення “вгору” при “середньому” значенні
>>> round(4.5)
4          # округлення “вниз(!?)” при “середньому” значенні
```

Працює правило “округлення, пов’язаного з парними числами” - тобто якщо цифра числа перед тим що округлюється парна - округлення робиться “вниз”, а якщо непарна - то вгору. Це рекомендація IEEE 754 яка направлена на те щоб обмежити вплив округлення на операції з великою кількістю чисел.

int&float: зв'язок з bool

ми вже працювали з “конструкторами” відповідних типів `str()` і `bool()` - які отримують якийсь об'єкт і намагаються створити з нього об'єкт свого класу - `str()` - повертає рядок, `bool()` - повертає `True/False` відповідно правил для кожного типу об'єктів:

```
>>> str(3.1415)
'3.1415'
```

```
>>> bool("it is my string")
True
```

Точно так же ми маємо “конструктори” типів `int()` та `float()` які отримують об'єкт і намагаються перетворити його на “свій” тип даних - якщо це можливо:

```
>>> int("23")
23
>>> float("4.56")
4.56
>>> int(True)
1
```

правило співставлення чисел (`float` та `int`) у тип `bool`: якщо число дорівнює 0 - то це `False`, в інших випадках це `True`

```
>>> bool(3.1415)
True
```

Самостійне опрацювання:

Напишіть скрипт, на початку якого Ви будете присвоювати двом змінним рядкові літерали, які повинні відповідати числам ("123", "78.77", "-32.67"...)
уявіть що ці числа вводить користувач і, таким чином, можуть бути допущені різного роду помилки при вводі

- пробіли на початку і в кінці
- кома замість крапки при вводі
- пробіл після знаку "-" перед числом
- знак "+" перед числом

Ви повинні за допомогою методів рядків попередньо обробити ці рядки щоб зменшити кількість помилок при перетворенні рядка в число.

Далі Ви повинні перетворити ці рядки на числа - float.

Після цього надрукувати результат суми цих чисел і їх різниці, відформатований за допомогою f-рядків, точність при виводі числа - два знаки після коми (міні-мова форматування).

Наступний матеріал: керуючи структури.